

# vennDiagramLab — R Package

Zoltán Dul, Márton Ölbei, N. Shaun B. Thomas, Azeddine Si Ammour, Attila Csikász-Nagy

v2.4.1 — 2026-06-10

## Contents

|                                                                      |          |
|----------------------------------------------------------------------|----------|
| <b>Overview</b>                                                      | <b>3</b> |
| <b>Installation</b>                                                  | <b>4</b> |
| <b>Quick start</b>                                                   | <b>4</b> |
| <b>Core concepts</b>                                                 | <b>5</b> |
| <b>Bundled sample datasets</b>                                       | <b>5</b> |
| <b>R API reference</b>                                               | <b>6</b> |
| I/O loaders                                                          | 6        |
| load_csv(path, binary = TRUE, delimiter = NULL,<br>prefix_cols = 1L) | 6        |
| load_tsv(path, binary = TRUE, prefix_cols = 1L)                      | 6        |
| load_gmt(path)                                                       | 6        |
| load_gmx(path)                                                       | 6        |
| load_sample(name)                                                    | 6        |
| Catalog                                                              | 7        |
| list_samples()                                                       | 7        |
| list_models()                                                        | 7        |
| Analysis                                                             | 7        |
| analyze(dataset, model = "auto")                                     | 7        |
| statistics(result)                                                   | 7        |
| effective_universe(result)                                           | 7        |
| Statistics (lower-level functions)                                   | 8        |
| Item share distribution & cluster ordering                           | 8        |
| item_share_distribution(matrix)                                      | 8        |
| cluster_set_order(D, linkage = "average")                            | 9        |
| SVG renderers                                                        | 9        |

|                                                                                                                                                                                                           |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| render_venn_svg(result, model = NULL, set_names =<br>NULL, colors = NULL, title = NULL, show_names =<br>TRUE, show_counts = TRUE, show_items = FALSE,<br>item_options = NULL, highlight = NULL) . . . . . | 9  |
| render_share_distribution(dataset, style = NULL) . . . .                                                                                                                                                  | 10 |
| render_cluster_heatmap(result, linkage = "average",<br>show_row_dendrogram = TRUE, show_col_dendrogram<br>= TRUE, dendrogram_fraction = 0.12) . . . . .                                                   | 10 |
| render_enrichment_bar(result, metric = c("neglog10fdr",<br>"foldEnrichment"), width = 560L, height = 240L) .                                                                                              | 10 |
| render_enrichment_lollipop(result, metric = c("neglog10fdr",<br>"foldEnrichment"), width = 560L, height = 240L) .                                                                                         | 10 |
| generate_proportional_svg(result, width = 600L,<br>height = 600L) . . . . .                                                                                                                               | 10 |
| ggplot2 / ComplexUpset / ggraph renderers . . . . .                                                                                                                                                       | 10 |
| render_upset(result, max_columns = 20L, sort_by =<br>c("size", "degree"), threshold = 0L, color_mode<br>= c("depth", "heatmap", "custom"), colors = NULL) .                                               | 11 |
| render_network(result, edge_metric = "intersection",<br>seed = 42L, significance_threshold = 0.05,<br>node_color_map = NULL) . . . . .                                                                    | 11 |
| Reports . . . . .                                                                                                                                                                                         | 11 |
| to_pdf_report(result, path, title = NULL, include_network<br>= TRUE, include_about = TRUE, include_share =<br>TRUE, include_cluster = FALSE) . . . . .                                                    | 11 |
| to_excel_workbook(result, path) . . . . .                                                                                                                                                                 | 12 |
| to_zip_report(result, path, include_share = TRUE,<br>include_cluster = FALSE) . . . . .                                                                                                                   | 12 |
| TSV exports . . . . .                                                                                                                                                                                     | 12 |
| Proportional layout . . . . .                                                                                                                                                                             | 12 |
| solve_2set(a_only, b_only, ab) . . . . .                                                                                                                                                                  | 12 |
| solve_3set(regions) . . . . .                                                                                                                                                                             | 13 |
| circle_intersection_area(r1, r2, d) . . . . .                                                                                                                                                             | 13 |
| S4 classes . . . . .                                                                                                                                                                                      | 13 |
| Region accessors and Boolean DSL . . . . .                                                                                                                                                                | 14 |
| intersection_items(result, sets) . . . . .                                                                                                                                                                | 14 |
| exclusive_items(result, sets) . . . . .                                                                                                                                                                   | 14 |
| union_items(result, sets) . . . . .                                                                                                                                                                       | 14 |
| parse_region_expression(expr, n_sets) . . . . .                                                                                                                                                           | 15 |
| Composability — full pipeline . . . . .                                                                                                                                                                   | 15 |
| ggplot2 layer (R-only) . . . . .                                                                                                                                                                          | 16 |
| geom_venn(mapping = NULL, data = NULL, stat = "venn",<br>position = "identity", ..., width_px = 480) . . .                                                                                                | 16 |
| broom tidiers (R-only) . . . . .                                                                                                                                                                          | 16 |

## Vignettes

16

|                                                                                      |           |
|--------------------------------------------------------------------------------------|-----------|
| <b>Common workflows</b>                                                              | <b>17</b> |
| End-to-end PDF report from a TSV . . . . .                                           | 17        |
| Full bundle (PDF + SVGs + TSVs + xlsx + README) for a down-<br>stream user . . . . . | 17        |
| Spotlight + item names for a presentation slide . . . . .                            | 17        |
| Significance enrichment in a tidyverse pipeline . . . . .                            | 17        |
| Custom 4-letter colour mapping in the Venn . . . . .                                 | 18        |
| CI parity test: compare two TSV exports byte-by-byte . . . . .                       | 18        |
| <b>Output formats</b>                                                                | <b>18</b> |
| <b>Cross-package parity</b>                                                          | <b>19</b> |
| <b>Troubleshooting</b>                                                               | <b>20</b> |
| <b>References</b>                                                                    | <b>21</b> |
| Citation . . . . .                                                                   | 22        |

## Overview

`vennDiagramLab` is the R companion to the *Venn Diagram Lab* web tool (<https://venndiagramlab.org/>) and its Python sibling. It exposes the same set-analysis math, the same 44 SVG models, the same PDF report layout, and a byte-equivalent TSV/Excel export contract — wrapped in idiomatic R: S4 classes, `ggplot2` layers, broom tidiers, and a CRAN-clean R CMD check.

The package is designed for three audiences:

- **Scripted analysis** — load a dataset, analyze it, produce a multi-page PDF report in three lines.
- **Bioconductor / ggplot2 users** — `geom_venn()` is a `ggplot2` stat-layer; `broom::tidy()`, `broom::glance()`, and `broom::augment()` return tibbles ready for tidyverse pipelines.
- **CRAN-clean pipelines** — every function ships an `@examples` block; the package passes R CMD check with 0 errors, 0 warnings, 0 notes.

Sister projects:

| Surface          | URL / Install                                                         | Notes                                       |
|------------------|-----------------------------------------------------------------------|---------------------------------------------|
| Web tool         | <a href="https://venndiagramlab.org/">https://venndiagramlab.org/</a> | Browser SVG editor + interactive analysis   |
| Python companion | <code>pip install venn-diagram-lab</code>                             | Headless Python lib + vdl CLI               |
| R (this)         | <code>install.packages("vennDiagramLab")</code>                       | <code>vennDiagramLab2</code> flavoured port |

All three are kept in lock-step at the same major version.

## Installation

From CRAN (recommended):

```
install.packages("vennDiagramLab")
```

From GitHub (latest dev):

```
# install.packages("remotes")
remotes::install_github("ZoliQua/Venn-Diagram-Lab", subdir = "r")
```

Or, if cloned locally for hacking:

```
devtools::load_all("/path/to/Venn-Diagram-Lab/r")
```

Verify:

```
library(vennDiagramLab)
packageVersion("vennDiagramLab") # '2.4.1'
vdl_version()                     # "2.4.1"
```

Hard requirements (pulled in automatically): `methods`, `stats`, `utils`, `jsonlite`, `xml2`, `ggplot2`, `ComplexUpset`, `ggraph`, `tidygraph`, `grDevices`, `rsvg`, `patchwork`, `gridExtra`, `openxlsx`, `zip`, `BiocGenerics`.

For ZIP-bundle SVGs of the UpSet / Network plots, the optional `svglite` package is preferred but not required (a `cairo` / `base-PNG` fallback is used when `svglite` is absent).

## Quick start

```
library(vennDiagramLab)
```

```
ds <- load_sample("dataset_real_cancer_drivers_4")
res <- analyze(ds) # 4 sets -> 15 regions + statistics

to_pdf_report(res, "report.pdf") # 7-9 page PDF
statistics(res)@jaccard          # 4x4 matrix
```

Or, in a single pipe:

```
"data/genes.tsv" |>
  load_tsv() |>
  analyze() |>
  to_pdf_report("output/report.pdf")
```

## Core concepts

| Concept              | S4 class                      | Holds                                                                                                                  |
|----------------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Dataset</b>       | <code>VennDataset</code>      | Set names, item lists, item order, universe size, source metadata                                                      |
| <b>Region result</b> | <code>RegionResult</code>     | Dataset + per-region item lists + per-region masks + statistics                                                        |
| <b>Region</b>        | <code>RegionData</code>       | One region: bitmask, label ("AB"), exclusive items, inclusive items                                                    |
| <b>Statistics</b>    | <code>StatisticsResult</code> | Square matrices for jaccard / dice / overlap / fold_enrichment + long-form hypergeometric data.frame                   |
| <b>SVG image</b>     | <code>SvgImage</code>         | <code>content: character</code> , <code>width: integer</code> , <code>height: integer</code> returned by SVG renderers |

S4 slot access uses @:

```
res <- analyze(load_sample("dataset_real_cancer_drivers_4"))
res@dataset@set_names           # c("Vogelstein", "COSMIC_CGC", ...)
res@regions[["7"]@exclusive_items # items in A & B & C only
statistics(res)@jaccard["A", "B"] # one cell of the matrix
```

A **region** is one of the  $2^n - 1$  non-empty subsets of an n-set diagram. Each region has:

- a **bitmask** — 1 for set A only, 2 for set B only, 3 for A & B, 7 for A & B & C, etc.
- a **label** — the concatenation of member letters, e.g. "A", "AB", "ABC"
- **exclusive items** — items in this region and *only* this region
- **inclusive items** — items in this region OR any of its supersets

## Bundled sample datasets

Five curated datasets ship with the package (3 real biological, 2 mock). All are 4-set TSVs (binary item-by-set matrices), suitable for every public function:

```
list_samples()
# [1] "dataset_real_cancer_drivers_4"
# [2] "dataset_real_kegg_pathway_genes_4"
# [3] "dataset_real_mirna_targets_4"
# [4] "dataset_mock_overlap_4"
# [5] "dataset_mock_disjoint_4"

ds <- load_sample("dataset_real_cancer_drivers_4")
```

## R API reference

### I/O loaders

All five loaders return a `VennDataset` S4 object ready for `analyze()`.

```
load_csv(path, binary = TRUE, delimiter = NULL, prefix_cols = 1L)
```

Load a comma-separated input. With `binary = TRUE` (default) the file is a wide-form item  $\times$  set matrix with 0/1 cells; with `binary = FALSE` items are listed under set-named columns (one item per cell). `delimiter = NULL` auto-detects between `,` and `;`. `prefix_cols = 1L` means the first column is the item ID — set higher for files with multiple identifier columns.

```
ds <- load_csv("genes.csv", binary = TRUE, prefix_cols = 1L)
```

```
load_tsv(path, binary = TRUE, prefix_cols = 1L)
```

Same contract as `load_csv()` but tab-separated (the canonical webtool format).

```
ds <- load_tsv("genes.tsv")
```

```
load_gmt(path)
```

Load a Broad GMT file (tab-separated, one set per line: `set_name<TAB>description<TAB>item1<TAB>item2...`). Common in MSigDB-style gene-set distributions.

```
load_gmx(path)
```

Load a Broad GMX file (column-oriented sets, row 1 = set names, row 2 = descriptions, remaining rows = items).

```
load_sample(name)
```

Load one of the bundled curated demo datasets by stem name (no path, no extension).

```
for (name in list_samples()) {
  ds <- load_sample(name)
```

```

    cat(name, "->", length(ds@set_names), "sets,", length(ds@item_order), "items\n")
  }

```

## Catalog

**list\_samples()**

Return the names of bundled demo datasets in deterministic order.

**list\_models()**

Return a data.frame describing all 44 bundled Venn-diagram models. Columns: **name** (filename stem), **n\_sets** (2-9), **family** (Venn / Edwards / Anderson / ...).

```

models <- list_models()
head(models, 5)
#           name n_sets family
# 1   venn-2-set      2   Venn
# 2   venn-3-set      3   Venn
# 3   venn-4-set      4   Venn
# 4 venn-4a-set-edwards  4 Edwards
# 5 venn-4b-set-anderson  4 Anderson

```

## Analysis

**analyze(dataset, model = "auto")**

Compute all  $2^n - 1$  regions plus pairwise statistics. With **model = "auto"** the canonical Venn model for the set count is picked (**venn-2-set**, **venn-3-set**, **venn-4-set**, ...). Pass an explicit model name (e.g. **"venn-5a-set-edwards"**) to render with a different layout — the math is layout-independent.

```

res <- analyze(ds)
length(res@regions)           # 15 (for 4 sets)
statistics(res)$jaccard        # 4x4 matrix
statistics(res)$hypergeometric # data.frame: set_a, set_b, intersection,
                                # expected, p_value, p_adjusted, significance
res@regions[["7"]$exclusive_items # items in A & B & C only

```

**statistics(result)**

Generic accessor (S4) returning the **StatisticsResult** slot of a **RegionResult**. Same content as **result@statistics**, but the function form is the idiomatic R API.

**effective\_universe(result)**

Return the universe size used by the hypergeometric test for **result** (either the dataset's explicit **universe\_size**, or the union of all item lists when the dataset

omits one).

## Statistics (lower-level functions)

`compute_pairwise()` is what `analyze()` calls internally; the rest are scalar building blocks you can use independently in test code or custom pipelines.

| Function                                                                                         | Returns                       | Notes                                                     |
|--------------------------------------------------------------------------------------------------|-------------------------------|-----------------------------------------------------------|
| <code>jaccard(size_a, size_b, intersection)</code>                                               | numeric in [0,1]              | $ A \& B  /  A \text{ or } B $ , 0 when both sets empty   |
| <code>dice(size_a, size_b, intersection)</code>                                                  | numeric in [0,1]              | $2 A \& B  / ( A  +  B )$ , the Sorensen-Dice coefficient |
| <code>overlap_coefficient(size_a, size_b, intersection)</code>                                   | numeric in [0,1]              | $ A \& B  / \min( A ,  B )$ , Szymkiewicz-Simpson         |
| <code>hypergeometric_p_value(K, n, k)</code>                                                     | numeric in [0,1]              | One-sided over-representation $P(X \geq k)$               |
| <code>fold_enrichment(N, K, n, k)</code>                                                         | numeric $\geq 0$              | $(k \cdot N) / (K \cdot n)$ , observed / expected         |
| <code>bh_fdr(p_values)</code>                                                                    | numeric vector                | Benjamini-Hochberg adjusted p-values                      |
| <code>compute_pairwise(set_names, inclusive_sizes, pairwise_intersections, universe_size)</code> | <code>StatisticsResult</code> | Roll-up of all of the above into matrices + a long table  |

```
# Hypergeometric: are 126 shared items unusual when set A has 138, B has 593, universe is 20000
hypergeometric_p_value(20000, 138, 593, 126) # ~6.75e-184 (highly enriched)
fold_enrichment(20000, 138, 593, 126) # ~30.8 fold over expected
bh_fdr(c(0.01, 0.04, 0.20, 0.50)) # c(0.04, 0.08, 0.267, 0.50)
```

## Item share distribution & cluster ordering

`item_share_distribution(matrix)`

Take a binary item  $\times$  set matrix; return a named integer vector `c("1" = ..., "2" = ...)` where the name is the number of sets each item belongs to. Tall low-k bar = set-specific signal dominates; tall high-k bar = a core shared across every set.

```
m <- rbind(c(1,1,0), c(1,1,1), c(0,1,1), c(1,0,0))
colnames(m) <- c("A", "B", "C")
item_share_distribution(m) # 1: 1, 2: 2, 3: 1
```



```
cluster_set_order(D, linkage = "average")
```

Hierarchical agglomerative clustering on a symmetric distance matrix ( $D[i,j] \geq 0$ ,  $D[i,i] = 0$ ). Methods: "average" (UPGMA, default), "complete", "single". Returns a list with `leaf_order` (left-to-right ordering of original indices) and `merges` (N-1-row data.frame with `left`, `right`, `height`, `size`). Same ordering powers the cluster-heatmap renderer.

## SVG renderers

All seven SVG renderers return an `SvgImage S4` object. Access the raw SVG string via `img@content` (or `slot(img, "content")`); save with `writeLines(img@content, "out.svg")`.

```
render_venn_svg(result, model = NULL, set_names = NULL, colors
= NULL, title = NULL, show_names = TRUE, show_counts = TRUE,
show_items = FALSE, item_options = NULL, highlight = NULL)
```

Render the 44-model Venn template, substituting set names, counts, colours, and the optional title. `model = NULL` uses `result@model`. `set_names` overrides per-letter display names (e.g. `c(A = "Sample A")`). `colors` overrides per-letter fills (e.g. `c(A = "#FFF200")`). `show_names = FALSE` blanks the labels; `show_counts = FALSE` blanks the numbers.

### Item display and spotlight parameters:

- `show_items = TRUE` replaces each region's count with the actual item identifiers rendered as multi-line `<tspan>` content inside the existing `Count_*` text nodes. Pair with `item_options` to tune the layout.
- `item_options` is a named list. Recognised keys (with defaults): `max_items_per_region = 20L`, `ncol_items = 1L`, `truncate_long_names = 12L` (0 disables), `line_height = 10L`, `font_size = 8L`, `show_counts_with_items = FALSE`, `ellipsis = "..."`. Unknown keys raise a warning. When a region has more items than `max_items_per_region`, the excess is summarised as an italic "+N more" line.
- `highlight` accepts either a character vector of region labels (e.g. `c("AB", "ABC")`) or an integer vector of region bitmasks (e.g. the return value of `parse_region_expression()`, see the dedicated section below). Sets whose bit is set in at least one highlighted region keep their original fill; sets that do not contribute to any highlighted region are desaturated to #cccccc at 25% opacity. `NULL` (default) disables the spotlight.

```
img <- render_venn_svg(res, title = "Cancer drivers (4 sources)")
writeLines(img@content, "venn.svg")
```

```
# Item names inside regions, truncated to 10 characters, max 8 per region.
img <- render_venn_svg(res, show_items = TRUE,
  item_options = list(max_items_per_region = 8L,
```

```

truncate_long_names = 10L))

# Spotlight on regions matching a Boolean DSL expression.
masks <- parse_region_expression("A & B & ~C & ~D", n_sets = 4L)
img <- render_venn_svg(res, highlight = masks, show_items = TRUE)

render_share_distribution(dataset, style = NULL)

A 480x280 viewBox histogram. Tier-gradient fill from #ffe4b5 (k=1) to
#7e14ff (k=N). Used as a page in the PDF report when include_share =
TRUE (default).

render_cluster_heatmap(result, linkage = "average", show_row_dendrogram
= TRUE, show_col_dendrogram = TRUE, dendrogram_fraction = 0.12)

Cluster-ordered Jaccard heatmap with L-shaped dendrograms. dendrogram_fraction
controls how much of the canvas each dendrogram band gets (min 20 px effective).

render_enrichment_bar(result, metric = c("neglog10fdr", "foldEnrichment"),
width = 560L, height = 240L)

Pairwise enrichment bar chart. metric is "neglog10fdr" (default,
-log10(BH-FDR)) or "foldEnrichment". Green (#2e7d32) for FDR <
0.05, grey (#888888) otherwise. Significance markers (***) < 0.001, ** < 0.01,
* < 0.05) above each bar.

render_enrichment_lollipop(result, metric = c("neglog10fdr",
"foldEnrichment"), width = 560L, height = 240L)

Same data + same colours as the bar chart, drawn as a stem-and-dot plot where
the dot radius scales as sqrt(intersection / max_intersection) (range 2.5-
8 px). Useful when you want to encode TWO numbers per pair (significance ×
overlap size) in one glyph.

generate_proportional_svg(result, width = 600L, height = 600L)

Area-proportional circles for 2- and 3-set inputs (analytic solve for n=2, binary
search + sf::st_union() for n=3). Returns the raw SVG string. For 4+ sets the
geometry problem is ill-posed; use a non-proportional model instead.

```

## ggplot2 / ComplexUpset / ggraph renderers

Two of the seven renderers return a ggplot object instead of an SVG string. Save with `ggplot2::ggsave()` or display in an interactive session.

```
render_upset(result, max_columns = 20L, sort_by = c("size",
"degree"), threshold = 0L, color_mode = c("depth", "heatmap",
"custom"), colors = NULL)
```

UpSet plot via the `ComplexUpset` package: top bars = intersection sizes, left bars = set sizes, central matrix = membership dots. `sort_by` is "size" (default) or "degree". `color_mode` is "depth" (viridis on membership-degree), "heatmap" (Reds on size), or "custom" (use the `colors` map). `threshold` excludes intersections below the given size.

```
p <- render_upset(res, max_columns = 10L, sort_by = "degree", color_mode = "heatmap")
ggplot2::ggsave("upset.png", p, width = 8, height = 5, dpi = 200)
```

NB: `ComplexUpset` 1.3.x + `ggplot2` 4.0+ interact badly with the `S7` dispatch introduced in R 4.6+. On R < 4.6 you'll see a warning at first use; PDF/ZIP reports auto-skip the UpSet integration tests in that environment.

```
render_network(result, edge_metric = "intersection", seed = 42L,
significance_threshold = 0.05, node_color_map = NULL)
```

Force-directed network via `ggraph` + `tidygraph`: nodes = sets (radius proportional to size), edges = pairwise overlaps. `edge_metric` is one of "intersection", "jaccard", "fold\_enrichment", "overlap\_coefficient" and controls edge thickness. Edges below `significance_threshold` (BH-FDR) are grey; significant edges are blue (`#2E3192`). `node_color_map = c(A = "#FFF200")` overrides per-letter node fills.

## Reports

```
to_pdf_report(result, path, title = NULL, include_network = TRUE,
include_about = TRUE, include_share = TRUE, include_cluster =
FALSE)
```

The flagship multi-page PDF report. Pages produced (in order):

1. Overview (metadata + per-set size table)
2. Venn + UpSet side-by-side (Venn at native 1:1 aspect; ~218 dpi)
3. Statistics (Jaccard, Dice, Enrichment tables — paginated when set count > 4)
4. Item Share Distribution (always on, opt-out with `include_share = FALSE`)
5. Cluster Heatmap (opt-in via `include_cluster = TRUE`)
6. Network + significant edges list (skip with `include_network = FALSE`)
7. About / Credits / Cite (skip with `include_about = FALSE`)

```
to_pdf_report(res, "report.pdf",
title = "My analysis", include_cluster = TRUE)
```

```
to_excel_workbook(result, path)
```

Three-sheet Excel workbook matching the *Download Everything* ZIP from the webtool:

- **Jaccard** —  $N \times N$  matrix of Jaccard indices
- **Sorensen-Dice** —  $N \times N$  matrix of Dice coefficients (the actual sheet title uses the o-with-stroke character)
- **Enrichment** — long-form `set_a`, `set_b`, `intersection`, `union`, `expected`, `fold_enrichment`, `p_value`, `fdr`, `significant`

Implementation uses `openxlsx` (pure R, no Java).

```
to_excel_workbook(res, "stats.xlsx")
```

```
to_zip_report(result, path, include_share = TRUE, include_cluster = FALSE)
```

Full Report ZIP bundle: PDF + 4 SVGs + 3 TSVs + xlsx + README.txt, mirroring the webtool's *Download Everything* button. README header carries provenance (timestamp, package version, dataset, model) + a methodology section (re-used from the PDF appendix).

```
to_zip_report(res, "bundle.zip", include_cluster = TRUE)
```

### TSV exports

| Function                                         | Output                                                    |
|--------------------------------------------------|-----------------------------------------------------------|
| <code>to_region_summary_tsv(result, path)</code> | Per-region exclusive + inclusive counts                   |
| <code>to_matrix_tsv(result, path)</code>         | Binary item $\times$ set membership table                 |
| <code>to_statistics_tsv(result, path)</code>     | Long-form pairwise table (Jaccard / Dice / OC / FE / FDR) |

The output bytes match the Python sibling byte-for-byte (parity-tested in the test suite).

### Proportional layout

```
solve_2set(a_only, b_only, ab)
```

Analytical 2-circle solver. `a_only` and `b_only` are the exclusive counts; `ab` is the intersection. Returns a list with centre coordinates + radii so the rendered areas match the input ratios.

`solve_3set(regions)`

Three-circle solver. `regions` is the bitmask-keyed exclusive-count list (`list("1" = a_only, "2" = b_only, "3" = ab, "4" = c_only, "5" = ac, "6" = bc, "7" = abc)`). Uses binary search to converge on radii / centres.

`circle_intersection_area(r1, r2, d)`

Lens area for two circles of radius `r1`, `r2` whose centres are at distance `d`. Returns 0 when disjoint,  $\pi * \min(r1, r2)^2$  when fully contained.

## S4 classes

| Class            | Module         | Key slots                                                                                                                                                           |
|------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VennDataset      | vennDiagramLab | set_names: character,<br>items: list,<br>item_order:<br>character,<br>universe_size:<br>integer or NULL,<br>source_path:<br>character or NULL,<br>format: character |
| RegionResult     | vennDiagramLab | dataset: VennDataset,<br>model: character,<br>regions: list of<br>RegionData,<br>set_sizes: list,<br>statistics:<br>StatisticsResult                                |
| RegionData       | vennDiagramLab | mask: integer, label:<br>character,<br>set_indices: integer,<br>exclusive_items:<br>character,<br>inclusive_items:<br>character                                     |
| StatisticsResult | vennDiagramLab | jaccard, dice,<br>overlap_coefficient,<br>fold_enrichment (all 4<br>NxN matrices),<br>hypergeometric:<br>data.frame long-form                                       |

|          |                |                                                           |
|----------|----------------|-----------------------------------------------------------|
| SvgImage | vennDiagramLab | content: character,<br>width: integer,<br>height: integer |
|----------|----------------|-----------------------------------------------------------|

---

RegionResult also exposes S4 methods (used via generic dispatch):

```

statistics(result)                # StatisticsResult
effective_universe(result)        # integer
to_pdf_report(result, "report.pdf")
to_region_summary_tsv(result, "...")
to_matrix_tsv(result, "...")
to_statistics_tsv(result, "...")
to_excel_workbook(result, "...")
to_zip_report(result, "...")

```

## Region accessors and Boolean DSL

Four composable helpers for selecting items and regions inside an analysed diagram. All four are pure R, no new dependencies, and chain naturally with the existing renderers.

**intersection\_items(result, sets)**

Items appearing in **every** set listed in **sets**, regardless of whether they also appear in other (unlisted) sets. **sets** is a character vector of set letters ("A", "B", ...) or display names (the values of `result@dataset@set_names`); the two forms may be mixed.

```

res <- analyze(load_sample("dataset_real_cancer_drivers_4"))
intersection_items(res, c("Vogelstein", "COSMIC_CGC", "OncoKB"))
#> 126 items shared by these three sources (regardless of IntOGen membership)

```

**exclusive\_items(result, sets)**

Items in **exactly** this combination — in every set in **sets** AND in none of the other sets in the dataset. Equivalent to looking up the bitmask region and reading its `exclusive_items` slot.

```

exclusive_items(res, c("Vogelstein", "COSMIC_CGC"))
#> items in Vogelstein and COSMIC_CGC, but NOT in OncoKB or IntOGen

```

**union\_items(result, sets)**

Items appearing in **any** of the named sets (deduplicated, first-seen order). Equivalent to `unique(c(items_in_A, items_in_B, ...))`.

```

union_items(res, c("Vogelstein", "COSMIC_CGC"))

```

### `parse_region_expression(expr, n_sets)`

Small Boolean expression parser. Translates an expression like "A & B + !C" into a sorted integer vector of region bitmasks (each  $1..2^{n\_sets} - 1$ ). Designed to compose with `render_venn_svg()`'s `highlight = ...` parameter.

Grammar (precedence highest first):

| Operator                         | Meaning            | Example                                                        |
|----------------------------------|--------------------|----------------------------------------------------------------|
| <code>~</code> or <code>!</code> | Complement (unary) | <code>~A</code> matches regions where A is NOT a member        |
| <code>&amp;</code>               | Intersection       | <code>A &amp; B</code> matches regions containing both A and B |
| <code> </code> or <code>+</code> | Union              | <code>A + B</code> matches regions containing A or B           |
| <code>(, )</code>                | Grouping           | <code>A &amp; (B + C)</code>                                   |

Atoms are uppercase ASCII letters A..I (one per set).

```
parse_region_expression("A & B", n_sets = 3L)
#> [1] 3 7
parse_region_expression("A + B", n_sets = 3L)
#> [1] 1 2 3 5 6 7
parse_region_expression("A & ~B", n_sets = 3L)
#> [1] 1 5
parse_region_expression("(A | B) & C", n_sets = 3L)
#> [1] 5 6 7
```

### Composability — full pipeline

```
res <- analyze(load_sample("dataset_real_cancer_drivers_4"))

# Spotlight: highlight the regions where the first three sources agree
# (exclusively of IntOGen) plus the all-four core.
masks <- parse_region_expression("A & B & C", n_sets = 4L)
img <- render_venn_svg(res, highlight = masks, show_items = TRUE,
  item_options = list(max_items_per_region = 8L))
writeLines(img@content, "highlighted.svg")

# Extract the gene list for one highlighted region.
exclusive_items(res, c("Vogelstein", "COSMIC_CGC", "OncoKB"))
```

## ggplot2 layer (R-only)

```
geom_venn(mapping = NULL, data = NULL, stat = "venn", position =  
"identity", ..., width_px = 480)
```

A ggplot2 stat-layer that rasterises a `RegionResult` (or a precomputed list-column of one) into a Venn glyph inside a ggplot. Use within a tidyverse pipeline that faceted-plots multiple analyses side-by-side.

```
library(ggplot2)  
library(vennDiagramLab)  
  
ds <- load_sample("dataset_real_cancer_drivers_4")  
res <- analyze(ds)  
  
ggplot(data.frame(x = 1, y = 1, result = I(list(res)))) +  
  geom_venn(aes(x = x, y = y, result = result), width_px = 600) +  
  theme_void()
```

## broom tidiers (R-only)

`vennDiagramLab` registers `tidy()`, `glance()`, and `augment()` methods on `RegionResult` so the tidyverse can consume an analysis result:

```
library(broom)  
library(vennDiagramLab)  
  
res <- analyze(load_sample("dataset_real_cancer_drivers_4"))  
tidy(res)      # one row per region: mask, label, exclusive_count, inclusive_count  
glance(res)    # one row: n_sets, n_regions, universe_size, model  
augment(res)   # one row per item: item, A, B, C, D (binary memberships)
```

## Vignettes

The package ships eight executed-during-R-CMD-check vignettes:

| Vignette                     | Topic                                                                           |
|------------------------------|---------------------------------------------------------------------------------|
| v01_quickstart               | 5-minute first-time tour                                                        |
| v02_real_cancer_drivers      | End-to-end analysis on a real biological dataset                                |
| v03_proportional_diagrams    | Area-proportional layouts via <code>solve_2set</code> / <code>solve_3set</code> |
| v04_upset_vs_venn_vs_network | When to use which visualization (decision matrix)                               |
| v05_statistics_deep_dive     | Hypergeometric, FDR correction, fold enrichment                                 |



| Vignette                      | Topic                                                         |
|-------------------------------|---------------------------------------------------------------|
| v06_pipeline_integration      | Snakemake / drake / targets recipes                           |
| v07_pdf_reports               | All <code>include_*</code> flags + custom titles              |
| v08_custom_styling_and_export | Custom palettes, set-name overrides,<br>TSV/Excel/ZIP exports |

Browse from R with:

```
browseVignettes("vennDiagramLab")
vignette("v01_quickstart", package = "vennDiagramLab")
```

## Common workflows

### End-to-end PDF report from a TSV

```
res <- analyze(load_tsv("data/genes.tsv"))
to_pdf_report(res, "output/report.pdf", include_cluster = TRUE)
```

### Full bundle (PDF + SVGs + TSVs + xlsx + README) for a downstream user

```
to_zip_report(res, "bundle.zip", include_cluster = TRUE)
```

### Spotlight + item names for a presentation slide

```
res <- analyze(load_sample("dataset_real_cancer_drivers_4"))
masks <- parse_region_expression("A & B & C & ~D + A & B & ~C & D", n_sets = 4L)
img <- render_venn_svg(res, highlight = masks, show_items = TRUE,
                       item_options = list(max_items_per_region = 8L,
                                           truncate_long_names = 10L))
writeLines(img@content, "slide_spotlight.svg")
```

### Significance enrichment in a tidyverse pipeline

```
library(dplyr)
library(vennDiagramLab)

res <- analyze(load_sample("dataset_real_cancer_drivers_4"))
statistics(res)@hypergeometric |>
  filter(significant) |>
  arrange(p_adjusted) |>
  head()
#   set_a      set_b intersection expected p_value p_adjusted significant
#   COSMIC_CGC OncoKB          581    35.76    0.0      0.0      TRUE
```

```
# COSMIC_CGC IntOGen 388 18.39 0.0 0.0 TRUE
# OncoKB IntOGen 477 38.96 0.0 0.0 TRUE
```

## Custom 4-letter colour mapping in the Venn

```
res <- analyze(load_sample("dataset_real_cancer_drivers_4"))
custom <- c(A = "#FF6B6B", B = "#4ECDC4", C = "#FFE66D", D = "#1A535C")
img <- render_venn_svg(res, colors = custom, title = "My palette")
writeLines(img@content, "custom.svg")
```

## CI parity test: compare two TSV exports byte-by-byte

```
to_statistics_tsv(res, "new.tsv")
tools::md5sum(c("baseline/stats.tsv", "new.tsv"))
# baseline/stats.tsv new.tsv
# "abc123..." "abc123..." # byte-identical
```

## Output formats

| Format                                                          | Surface                                                        | Spec                                                                                                        |
|-----------------------------------------------------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>TSV</b> (region-summary<br>/ matrix / statistics)            | <code>to_*_tsv()</code>                                        | UTF-8, LF line endings,<br>= / + / - / @ prefixes<br>escaped to prevent<br>CSV-injection in<br>spreadsheets |
| <b>SVG</b> (Venn / share-dist<br>/ heatmap / bar /<br>lollipop) | <code>render_*</code> returning<br><code>SvgImage</code>       | Plain SVG 1.1 viewBox,<br>no external CSS or<br>scripts, byte-equivalent<br>to the webtool's output         |
| <b>ggplot</b> (UpSet /<br>Network)                              | <code>render_upset()</code> ,<br><code>render_network()</code> | Save with<br><code>ggplot2::ggsave()</code> ;<br>raster (PNG) or vector<br>(SVG via <code>svglite</code> )  |
| <b>PDF</b> (report)                                             | <code>to_pdf_report()</code>                                   | US Letter landscape,<br>multi-page via<br><code>grDevices::pdf</code> +<br><code>patchwork</code>           |
| <b>Excel</b> (statistics<br>workbook)                           | <code>to_excel_workbook()</code> ,<br>embedded in ZIP          | 3 sheets: Jaccard,<br>Sorensen-Dice,<br>Enrichment; bold/grey<br>header rows + 4-decimal<br>numeric format  |

| Format                           | Surface                      | Spec                                                   |
|----------------------------------|------------------------------|--------------------------------------------------------|
| <b>ZIP</b> (Download Everything) | <code>to_zip_report()</code> | PDF + 4 SVGs + 3 TSVs + xlsx + README.txt, flat layout |

## Cross-package parity

This R package, the Python companion **venn-diagram-lab**, and the webtool all ship in lock-step. TSV exports are tested byte-equivalent across all three; PDF/SVG/PNG are functionally equivalent but pixel-level differences exist due to different rendering backends (grDevices vs matplotlib vs browser SVG).

| Functionality                                   | R                                                   | Python       | Webtool                    |
|-------------------------------------------------|-----------------------------------------------------|--------------|----------------------------|
| 5 loaders (CSV/TSV/GMT/GMX/sample)              | Yes                                                 | Yes          | CSV/TSV/GMT/GMX via dialog |
| 44 SVG models                                   | Yes                                                 | Yes          | Yes + interactive editor   |
| analyze + all statistics                        | Yes                                                 | Yes          | Yes                        |
| 7 SVG renderers                                 | Yes                                                 | Yes          | Yes                        |
| UpSet + Network (ggraph / matplotlib / browser) | Yes                                                 | Yes          | Yes                        |
| 4-page PDF report core                          | Yes                                                 | Yes          | Yes                        |
| Item Share                                      | Yes                                                 | Yes          | Yes                        |
| Distribution page                               | Yes (opt-in)                                        | Yes (opt-in) | Yes (toggle)               |
| Cluster Heatmap page                            | Yes (opt-in)                                        | Yes (opt-in) | Yes (toggle)               |
| ZIP bundle                                      | Yes                                                 | Yes          | Yes (Download Everything)  |
| Excel workbook                                  | Yes                                                 | Yes          | Yes                        |
| CLI ( <code>vd1 ...</code> )                    | No (R has no idiomatic CLI; the lib IS the surface) | Yes          | N/A                        |
| ggplot2                                         | Yes                                                 | No           | N/A                        |
| <code>geom_venn()</code>                        | Yes                                                 | No           | N/A                        |
| broom                                           | Yes                                                 | No           | N/A                        |
| <code>tidy()/glance()/augment()</code>          | Yes                                                 | No           | N/A                        |

| Functionality                                                                                                     | R   | Python                                            | Webtool |
|-------------------------------------------------------------------------------------------------------------------|-----|---------------------------------------------------|---------|
| In-region item display<br>( <code>show_items</code> )                                                             | Yes | No                                                | No      |
| Spotlight highlight<br>( <code>highlight = c("AB", ...)</code> )                                                  | Yes | No                                                | No      |
| Region accessors<br>( <code>intersection_items</code> / <code>exclusive_items</code> / <code>union_items</code> ) | Yes | No (use <code>RegionResult.regions[mask]</code> ) | No      |
| Boolean DSL<br>( <code>parse_region_expression</code> )                                                           | Yes | No                                                | No      |

## Troubleshooting

| Symptom                                                                                        | Cause                                                              | Remedy                                                                                                             |
|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Error: model has 5 sets, dataset has 4                                                         | Explicit model = "venn-5..." mismatches dataset's set count        | Use <code>model = "auto"</code> or pick a model with matching <code>n_sets</code>                                  |
| Warning<br>"ComplexUpset 1.3.3 ... incompatible"                                               | R < 4.6 + ggplot2 4 + S7 dispatch bug                              | Upgrade to R >= 4.6,<br>OR<br><code>remotes::install_github("krassowski/complex-upset")</code> for the dev version |
| Error: cannot open the connection during <code>devtools::document()</code>                     | DESCRIPTION Collate references a missing R file                    | Create the placeholder or remove the entry                                                                         |
| Warning "mbcsToSbcs: ... substituted for ..."                                                  | <code>grDevices::pdf</code> default Helvetica can't render em-dash | Cosmetic only; use ASCII <code>--</code> in source strings, or switch to a cairo-backed PDF                        |
| Error: paths are ignored by <code>.gitignore</code> when adding <code>docs/export_r.pdf</code> | The monorepo gitignores <code>docs/</code>                         | The artefact is intentional dev-tree noise — leave it untracked                                                    |

| Symptom                                                                    | Cause                                                                                                                                    | Remedy                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PDF report missing a page                                                  | <code>include_share = FALSE / include_network = FALSE / include_about = FALSE</code> , or <code>include_cluster = FALSE</code> (default) | Re-run without the flag, or pass <code>include_cluster = TRUE</code>                                                                                                                                                              |
| Error: Unknown set identifier: 'Z' from a region accessor                  | Set letter or display name does not match <code>result@dataset@set_names</code>                                                          | Use <code>result@dataset@set_names</code> to list valid identifiers; letters are A..I in dataset order                                                                                                                            |
| Error: highlight: no region with mask 99 from <code>render_venn_svg</code> | A bitmask outside <code>1..(2^n_sets - 1)</code> was passed to <code>highlight</code>                                                    | Use <code>parse_region_expression()</code> to obtain a valid mask vector, or check <code>1..(2^n - 1)</code> bounds                                                                                                               |
| Error: Malformed region expression: trailing operator                      | A Boolean DSL string ends with a dangling <code>&amp; / + / ~</code>                                                                     | Complete the expression — every operator needs operands on both sides                                                                                                                                                             |
| Error: Parenthesis mismatch in region expression                           | Unbalanced ( or ) in the Boolean DSL string                                                                                              | Recount opening vs. closing parentheses                                                                                                                                                                                           |
| Warning “Ignoring unknown item_options: ‘bogus’ ”                          | A typo in <code>item_options</code> keys                                                                                                 | Recognised keys: <code>max_items_per_region</code> , <code>ncol_items</code> , <code>truncate_long_names</code> , <code>line_height</code> , <code>font_size</code> , <code>show_counts_with_items</code> , <code>ellipsis</code> |

## References

- **Web tool:** <https://venndiagramlab.org/>
- **R package (CRAN):** <https://CRAN.R-project.org/package=vennDiagramLab>
- **Python sibling:** <https://pypi.org/project/venn-diagram-lab/>
- **GitHub monorepo:** <https://github.com/ZoliQua/Venn-Diagram-Lab>
- **R-package mirror (autosync):** <https://github.com/ZoliQua/vennDiagramLab>
- **Zenodo concept DOI:** <https://doi.org/10.5281/zenodo.19510813>

## Citation

Dul Z., Ölbei M., Thomas N.S.B., Si Ammour A., Csikász-Nagy A. (2026). *Venn Diagram Lab – Headless Venn diagram analysis and rendering*. R package version 2.4.1. <https://venndiagramlab.org/> doi:10.5281/zenodo.19510813